Chapter 1-Introduction to software engineering

Outline

- Introduction
- Two Orthogonal view of software.
- Software development process models
  - Software Process
  - Software life cycle and process models
- Object oriented system development methodology.
  - Why an object oriented
  - Overview of the unified approach.
  - An object oriented philosophy
  - concepts of an object
  - Attributes of an object, its state and properties.

What is Software Engineering?

- It is composed of two words, software and engineering.
- **Software** is defined as a collection of programs, procedures, rules, data and associated documentation.
  - The software is developed keeping in mind certain hardware and operating system consideration commonly known as platform.
- **Engineering** on the other hand, is all about developing products, using well-defined, scientific principles and methods.
- Software engineering is a branch of computer science that study how to develop a software. Which is the process of analysis, design, implementing , testing and maintaining software.
- Software characteristics are:
  - Software is custom built rather than assembled from existing components,
  - Software is developed or engineered not manufactured,
  - Software is flexible, and Software does not wear out.

SOFTWARE ENGINEERING OVERVIEW

- Software engineering is requires:
  1. **Teams**. The problems to solve are so complex or large, that a single developer cannot solve them anymore.
  2. **Communication**. Teams do not consist only of developers, but also of testers, architects, system engineers, customer, project managers, etc. so it requires effective communication with the teams.
  3. **Planning:** Software projects can be so large that we have to do careful.
  4. **Guidelines: Implementation** is no longer just writing code, but it is also following guidelines, writing documentation and also writing unit tests. But unit tests alone are not enough.
  5. **tools :** a good software engineer, like any engineer, needs tools, and you need to know about them.

1.1 TWO ORTHOGONAL VIEWS OF SOFTWARE:

- Object oriented methodologies Programs=**Algorithm** + **Data** Structures
- Orthogonality is one of the most important properties that can help make even complex designs compact.
- In a purely orthogonal design, operations do not have side effects; each changes just one thing without affecting others.
- Orthogonality is a software design principle for writing components in a way that changing one component doesn't affect other components.
- It is the combination of two other principles, namely strong cohesion and loose coupling.

Cont….

TRADITIONAL APPROACH	OBJECT ORIENTED SYSTEM DEVELOPMENT
Collection of procedures(functions)	Combination of data and functionality
Focuses on function and procedures, different styles and methodologies for each step of process	Focuses on object, classes, modules that can be easily replaced, modified and reused.
Moving from one phase to another phase is complex.	Moving from one phase to another phase is easier.
Increases duration of project	decreases duration of project
Increases complexity	Reduces complexity and redundancy

1.2. Software development process models

1.2.1. Software Engineering Development Activities

- A software process is the systematic approach that is used in software engineering to develop software. It is a set of activities that leads to the production of a software product.
- The fundamental Software Engineering Development Activities are:
  1. Requirements Elicitation
  2. Analysis
  3. System Design
  4. Object Design
  5. Implementation
  6. Testing

Requirements Elicitation:

- During requirements elicitation, the client and developers define the purpose of the system.
- The result of this activity is a description of the system in terms of actors and use cases.
- Actors represent the external entities that interact with the system. Actors include roles such as end users, other computers the system needs to deal with (e.g., a central bank computer, a network), and the environment (e.g., a chemical process).
- Use cases are general sequences of events that describe all the possible actions between an actor and the system for a given piece of functionality.

Analysis

- During analysis, developers aim to produce a model of the system that is correct, complete, consistent, and unambiguous.
- Developers transform the use cases produced during requirements elicitation into an object model that completely describes the system.
- During this activity, developers discover ambiguities and inconsistencies in the use case model that they resolve with the client.
- The result of analysis is a system model annotated with attributes, operations, and associations.
- The system model can be described in terms of its structure and its dynamic interoperation.

System Design

- During system design, developers define the design goals of the project and decompose the system into smaller subsystems that can be realized by individual teams.
- Developers also select strategies for building the system, such as the hardware/software platform on which the system will run, the persistent data management strategy, the global control flow, the access control policy, and the handling of boundary conditions.
- The result of system design is a clear description of each of these strategies, a subsystem decomposition, and a deployment diagram representing the hardware/software mapping of the system.
- Whereas both analysis and system design produce models of the system under construction, only analysis deals with entities that the client can understand.

Object Design

- During object design, developers define solution domain objects to bridge the gap between the analysis model and the hardware/software platform defined during system design.
- This includes precisely describing object and subsystem interfaces, selecting off-the-shelf components, restructuring the object model to attain design goals such as extensibility or understandability, and optimizing the object model for performance.
- The result of the object design activity is a detailed object model annotated with constraints and precise descriptions for each element.

Implementation

- During implementation, developers translate the solution domain model into source code.
- This includes implementing the attributes and methods of each object and integrating all the objects such that they function as a single system.
- The implementation activity spans the gap between the detailed object design model and a complete set of source code files that can be compiled

Testing

- During testing, developers find differences between the system and its models by executing the system (or parts of it) with sample input data sets.
- During unit testing, developers compare the object design model with each object and subsystem.
- During integration testing, combinations of subsystems are integrated together and compared with the system design model.
- During system testing, typical and exception cases are run through the system and compared with the requirements model.
- The goal of testing is to discover as many faults as possible such that they can be repaired before the delivery of the system.
- The planning of test phases occurs in parallel to the other development activities:

Cont…

- Sometimes, software processes are categorized as either plan-driven or agile processes.
  1. **plan-driven processes:** are processes where all of the process activities are planned in advance and progress is measured against this plan.
  2. **agile processes:** planning is incremental and it is easier to change the process to reflect changing customer requirements.
- Each approach is suitable for different types of software. Generally, you need to find a balance between plan-driven and agile processes.

1.2.2 Managing Software Development

- Managing a software engineering project are the activities focus on planning the project, monitoring its status, tracking changes, and coordinating resources such that a high-quality product is delivered on time and within budget.
- Management activities not only involve managers, but also most of the other project participants as well. Management activities include
  - Communication
  - Rationale Management
  - Software Configuration Management
  - Project Management
  - Software Life Cycle

Communication

- Communication is the most critical and time-consuming activity in software engineering. Misunderstandings and omissions often lead to faults and delays that are expensive to correct later in the development.
- Communication includes the exchange of models and documents about the system and its application domain, reporting the status of work products, providing feedback on the quality of work products, raising and negotiating issues, and communicating decisions.
- Communication is made difficult by the diversity of participants' backgrounds, by their geographic distribution, and by the volume, complexity, and evolution of the information exchanged.
- To deal with communication issues, project participants have many tools available.
- The most effective one is conventions: When participants agree on notations for representing information, on tools for manipulating information, and on procedures for raising and resolving issues, they already have eliminated substantial sources of misunderstanding.
- Examples of notations include UML diagrams, templates for writing documents and meeting minutes, and identification schemes for naming software components.

1.2.2. Software life cycle and process models

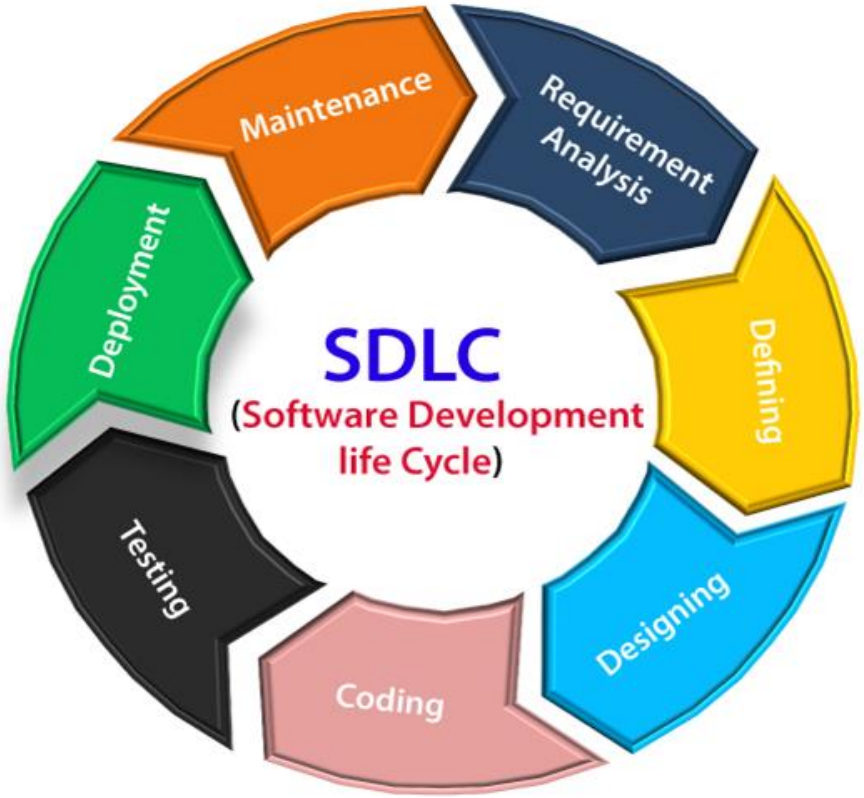
- A software life cycle model (also termed process model) is a pictorial and diagrammatic representation of the software life cycle.
- A life cycle model represents all the methods required to make a software product transit through its life cycle stages. It also captures the structure in which these methods are to be undertaken.
- In other words, a life cycle model maps the various activities performed on a software product from its inception to retirement.
- Different life cycle models may plan the necessary development activities to phases in different ways. Thus, no element which life cycle model is followed, the essential activities are contained in all life cycle models though the action may be carried out in distinct orders in different life cycle models. During any life cycle stage, more than one activity may also be carried out.

Cont….

- Need of SDLC
  - The development team must determine a suitable life cycle model for a particular plan and then observe to it.
  - Without using an exact life cycle model, the development of a software product would not be in a systematic and disciplined manner.
  - When a team is developing a software product, there must be a clear understanding among team representative about when and what to do. Otherwise, it would point to chaos and project failure.
  - Without software life cycle models, it becomes tough for software project managers to monitor the progress of the project

Cont…

- SDLC represents the process of developing software. SDLC framework includes the following steps (activities):



The stages of SDLC are as follows:

Stage1: Planning and Requirement elicitation

- Requirement elicitation is the most important and necessary stage in SDLC.
- The senior members of the team perform it with inputs from all the stakeholders and domain experts or SMEs in the industry.
- Planning for the quality assurance requirements and identifications of the risks associated with the projects is also done at this stage.
- Business analyst and Project organizer set up a meeting with the client to gather all the data like what the customer wants to build, who will be the end user, what is the objective of the product. Before creating a product, a core understanding or knowledge of the product is very necessary.

Cont..

- **For Example**, A client wants to have an application which concerns money transactions. In this method, the requirement has to be precise like what kind of operations will be done, how it will be done, etc.
- Once the required function is done, an analysis is complete with auditing the feasibility of the growth of a product.
- Once the requirement is understood, the SRS (Software Requirement Specification) document is created. The developers should thoroughly follow this document and also should be reviewed by the customer for future reference.

Cont..

Stage2: Defining Requirements /Analysis

- Once the requirement analysis is done, the next stage is to certainly represent and document the software requirements and get them accepted from the project stakeholders.
- This is accomplished through "SRS"- Software Requirement Specification document which contains all the product requirements to be constructed and developed during the project life cycle.

Stage3: Designing the Software / Designing

- The next phase is about to bring down all the knowledge of requirements, analysis, and design of the software project. This phase is the product of the last two, like inputs from the customer and requirement gathering.

Cont..

Stage4: Developing the project /implementation

- In this phase of SDLC, the actual development begins, and the programming is built. The implementation of design begins concerning writing code.
- Developers have to follow the coding guidelines described by their management and programming tools like compilers, interpreters, debuggers, etc. are used to develop and implement the code.

Stage5: Testing

- After the code is generated, it is tested against the requirements to make sure that the products are solving the needs addressed and gathered during the requirements stage.
- During this stage, unit testing, integration testing, system testing, acceptance testing are done.

Cont..

Stage6: Deployment

- Once the software is certified, and no bugs or errors are stated, then it is deployed.
- Then based on the assessment, the software may be released as it is or with suggested enhancement in the object segment.
- After the software is deployed, then its maintenance begins.

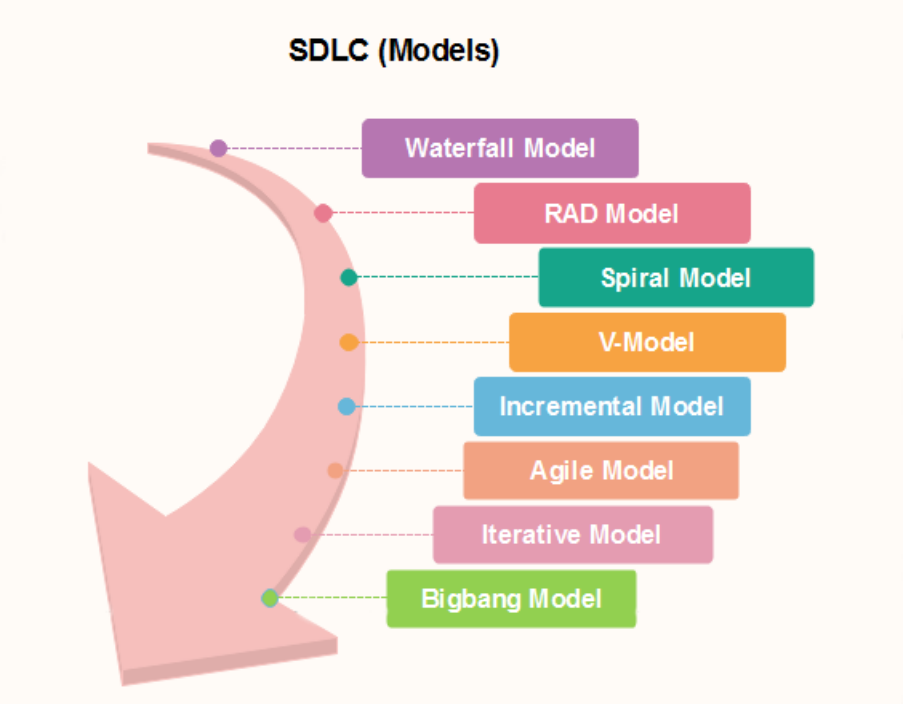
Stage7: Maintenance

- Once when the client starts using the developed systems, then the real issues come up and requirements to be solved from time to time.
- This procedure where the care is taken for the developed product is known as maintenance.

SDLC models or process models

- Software Development life cycle (SDLC) is a spiritual model used in project management that defines the stages include in an information system development project, from an initial feasibility study to the maintenance of the completed application.
- There are different software development life cycle models specify and design, which are followed during the software development phase.
- These models are also called "**Software Development Process Models.**" Each process model follows a series of phase unique to its type to ensure success in the step of software development.

Some of Software Development Process (SDLC) Models



Waterfall model

- The waterfall is a universally accepted SDLC model. The waterfall model is a continuous software development model in which development is seen as flowing steadily downwards (like a waterfall) through the steps.
- This model has five phases:
  1. Requirements analysis and specification phase:
  2. Design Phase:
  3. Implementation and unit testing:
  4. Integration and System Testing:
  5. Operation and maintenance phase:
- The steps always follow in this order and do not overlap. The developer must complete every phase before the next phase begins.

Cont..

Waterfall Model
Requirement Analysis and Specification
Design Phase
Implementation and Unit Testing
Integration and System Testing
Operation and maintenance phase

Cont..

- When to use SDLC Waterfall Model?
- Some Circumstances where the use of the Waterfall model is most suited are:
  - When the requirements are constant and not changed regularly.
  - A project is short
  - The situation is calm
  - Where the tools and technology used is consistent and is not changing
  - When resources are well prepared and are available to use.

Cont….

- Advantages of Waterfall model
  - This model is simple to implement also the number of resources that are required for it is minimal.
  - The requirements are simple and explicitly declared; they remain unchanged during the entire project development.
  - The start and end points for each phase is fixed, which makes it easy to cover progress.
  - The release date for the complete product, as well as its final cost, can be determined before development.
  - It gives easy to control and clarity for the customer due to a strict reporting system.

Cont..

- Disadvantages of Waterfall model
  - In this model, the risk factor is higher, so this model is not suitable for more significant and complex projects.
  - This model cannot accept the changes in requirements during development.
  - It becomes tough to go back to the phase. For example, if the application has now shifted to the coding phase, and there is a change in requirement, It becomes tough to go back and change it.
  - Since the testing done at a later stage, it does not allow identifying the challenges and risks in the earlier phase, so the risk reduction strategy is difficult to prepare.

Incremental Model

- Incremental Model is a process of software development where requirements divided into multiple standalone modules of the software development cycle.
- In this model, each module goes through the requirements, design, implementation and testing phases. Every subsequent release of the module adds function to the previous release. The process continues until the complete system achieved.

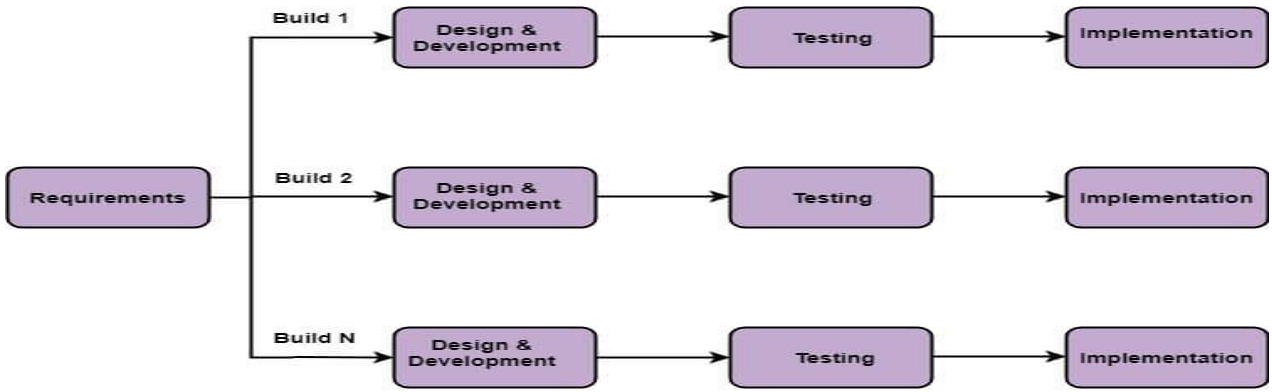


Fig: Incremental Model

The various phases of incremental model are as follows:

1. **Requirement analysis:** In the first phase of the incremental model, the product analysis expertise identifies the requirements. And the system functional requirements are understood by the requirement analysis team.
2. **Design & Development:** In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success.
3. **Testing:** In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality.
4. **Implementation:** Implementation phase enables the coding phase of the development system. It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product

Cont..

- When we use the Incremental Model?
  - When the requirements are superior.
  - A project has a lengthy development schedule.
  - When Software team are not very well skilled or trained.
  - When the customer demands a quick release of the product.
  - You can develop prioritized requirements first.
- Advantage of Incremental Model
  - Errors are easy to be recognized.
  - Easier to test and debug
  - More flexible.
  - Simple to manage risk because it handled during its iteration.
  - The Client gets important functionality early.
- Disadvantage of Incremental Model
  - Need for good planning
  - Total Cost is high.
  - Well defined module interfaces are needed.

Iterative Model

- In this Model, you can start with some of the software specifications and develop the first version of the software. After the first version if there is a need to change the software, then a new version of the software is created with a new iteration.
- Every release of the Iterative Model finishes in an exact and fixed period that is called iteration.
- The Iterative Model allows the accessing earlier phases, in which the variations made respectively. The final output of the project renewed at the end of the Software Development Life Cycle (SDLC) process.

Cont..

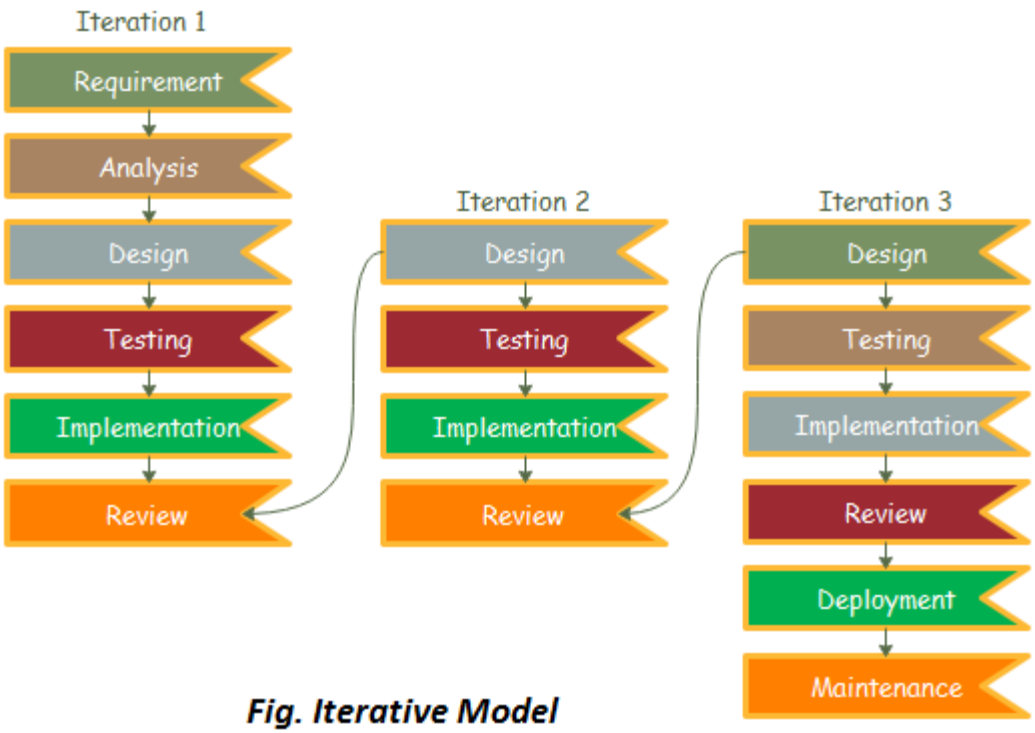


Fig. Iterative Model

The various phases of Iterative model are as follows:

1. **Requirement gathering & analysis:** In this phase, requirements are gathered from customers and check by an analyst whether requirements will fulfil or not. Analyst checks that need will achieve within budget or not. After all of this, the software team skips to the next phase.
2. **Design:** In the design phase, team design the software by the different diagrams like Data Flow diagram, activity diagram, class diagram, state transition diagram, etc.

- **3. Implementation:** In the implementation, requirements are written in the coding language and transformed into computer programmes which are called Software.

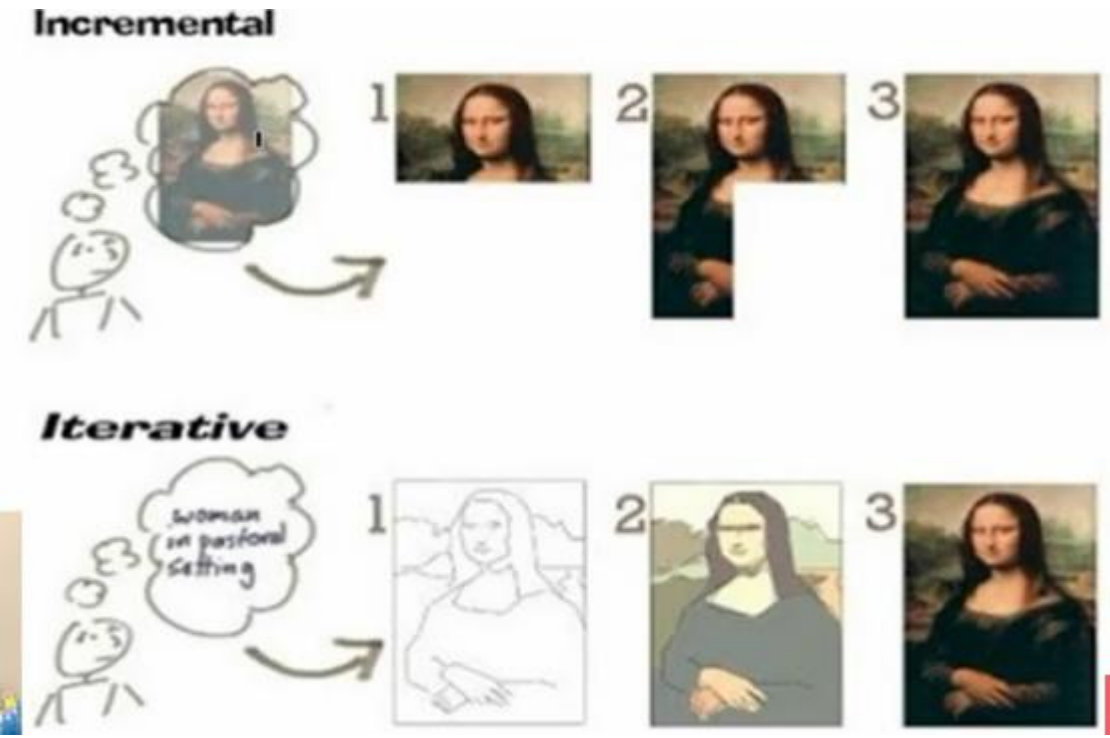
Cont..

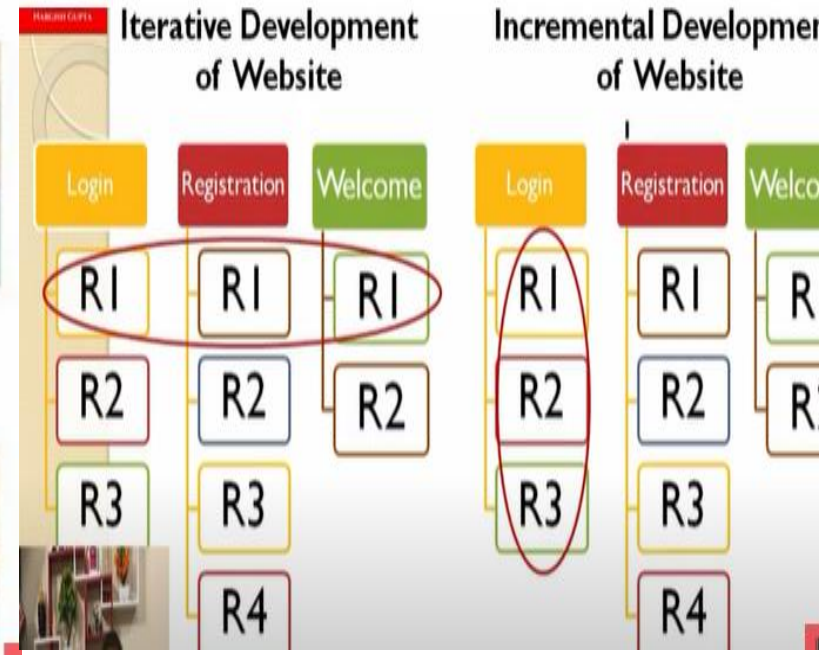
4. **Testing:** After completing the coding phase, software testing starts using different test methods. There are many test methods, but the most common are white box, black box, and grey box test methods.
5. **Deployment:** After completing all the phases, software is deployed to its work environment.
6. **Review:** In this phase, after the product deployment, review phase is performed to check the behaviour and validity of the developed product. And if there are any error found then the process starts again from the requirement gathering.
7. **Maintenance:** In the maintenance phase, after deployment of the software in the working environment there may be some bugs, some errors or new updates are required. Maintenance involves debugging and new addition options.

Cont..

- When to use the Iterative Model?
  - When requirements are defined clearly and easy to understand.
  - When the software application is large.
  - When there is a requirement of changes in future.
- Advantage(Pros) of Iterative Model:
  - Testing and debugging during smaller iteration is easy.
  - A Parallel development can plan.
  - It is easily acceptable to ever-changing needs of the project.
  - Risks are identified and resolved during iteration.
  - Limited time spent on documentation and extra time on designing.
- Disadvantage(Cons) of Iterative Model:
  - It is not suitable for smaller projects.
  - More Resources may be required.
  - Design can be changed again and again because of imperfect requirements.
  - Requirement changes can cause over budget.
  - Project completion date not confirmed because of changing requirements.

Difference between Incremental and Iterative



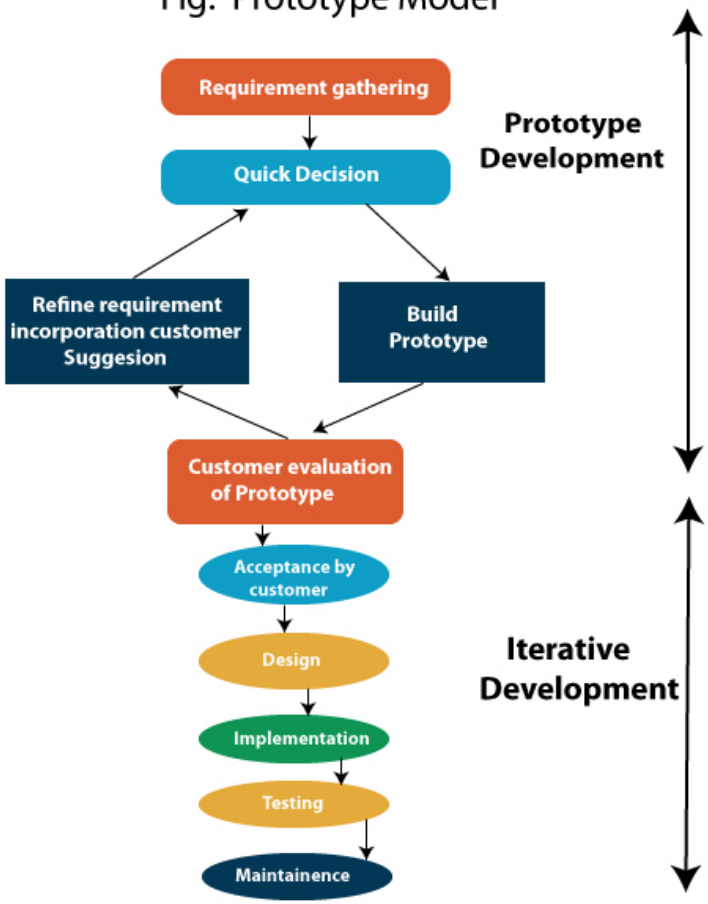


Prototype Model

- The prototype model requires that before carrying out the development of actual software, a working prototype of the system should be built.
- A prototype is a toy implementation of the system. A prototype usually turns out to be a very crude version of the actual system, possible exhibiting limited functional capabilities, low reliability, and inefficient performance as compared to actual software.
- In many instances, the client only has a general view of what is expected from the software product.
- In such a scenario where there is an absence of detailed information regarding the input to the system, the processing needs, and the output requirement, the prototyping model may be employed.

Cont..

Fig: Prototype Model



Cont….

- Steps of Prototype Model
  - Requirement Gathering and Analyst
  - Quick Decision
  - Build a Prototype
  - Assessment or User Evaluation
  - Prototype Refinement
  - Engineer Product
- Advantage of Prototype Model
  - Reduce the risk of incorrect user requirement
  - Good where requirement are changing/uncommitted
  - Regular visible process aids management
  - Support early product marketing
  - Reduce Maintenance cost.
  - Errors can be detected much earlier as the system is made side by side.

Cont..

- Disadvantage of Prototype Model
  - An unstable/badly implemented prototype often becomes the final product.
  - Require extensive customer collaboration
    - Costs customer money
    - Needs committed customer
    - Difficult to finish if customer withdraw
    - May be too customer specific, no broad market
  - Difficult to know how long the project will last.
  - Easy to fall back into the code and fix without proper requirement analysis, design, customer evaluation, and feedback.
  - Prototyping tools are expensive.
  - Special tools & techniques are required to build a prototype.
  - It is a time-consuming process.

Agile Model

- The meaning of Agile is swift or versatile. "**Agile process model**" refers to a software development approach based on iterative development.
- Agile methods break tasks into smaller iterations, or parts do not directly involve long term planning.
- The agile model refers to the iterative approach to delivering a software product. This means that instead of delivering a large project only when all parts are complete, a team breaks down a large project into smaller parts, and delivers these completed smaller parts in regular cycles.
  - For example, using the agile model a developer delivers multiple releases of a product and each release could add a single feature, such as drop-down menus, checkboxes, or multiple-choice buttons. The final release would contain all features.

Cont..

- Agile methods deal better with ill-defined requirements and emerging solution technologies, which are characterized by frequent change(by client, user, developer).
- Agile methods attempt to deal with these challenges by focusing on small product increments that are developed in short time bursts.
- After each product increment, progress is reviewed, lessons are learned, the project organization or goals are adjusted accordingly

Cont..

- Agile project management method uses **scrum** which is a management framework that teams use to self-organize and work towards a common goal. It describes a set of meetings, tools, and roles for efficient project delivery
- The Scrum approach is characterized by short iterations, called **sprints**, during which a release candidate is developed. In Scrum, this is called a potentially deliverable product increment, Consequently.
- The project is started with a project kick off meeting, during which the product owner and the Scrum master brainstorm a first version of the requirements. At the end of the kick off, the requirements are prioritized into a list called the product backlog.

Cont..

Scrum defines three roles:

- The **product owner** is responsible for the requirements. The product owner participates in the creation of the product backlog, and prioritizes the product backlog before each sprint.
- The **Scrum master** is a management role responsible for the process.
  - The Scrum master sets up and enforces the rules and practices of the project.
  - The Scrum master facilitates the daily Scrum meetings, monitors progress, and removes obstacles preventing the team to do its work.
  - The Scrum master is the interface between the Scrum team and the product owner, ensures the team productivity, and shields the team from distractions

Cont..

- **The Scrum team** is responsible for developing the product increment.
  - The team is cross functional including all the skills required to build the product increment.
  - The team is self organizing in that each member commits to work based on their own skills and availability.
  - Members learn and adapt their skills based on the work to be done.
  - The team participates in the sprint planning meeting and commits to the list of tasks defined in the sprint backlog

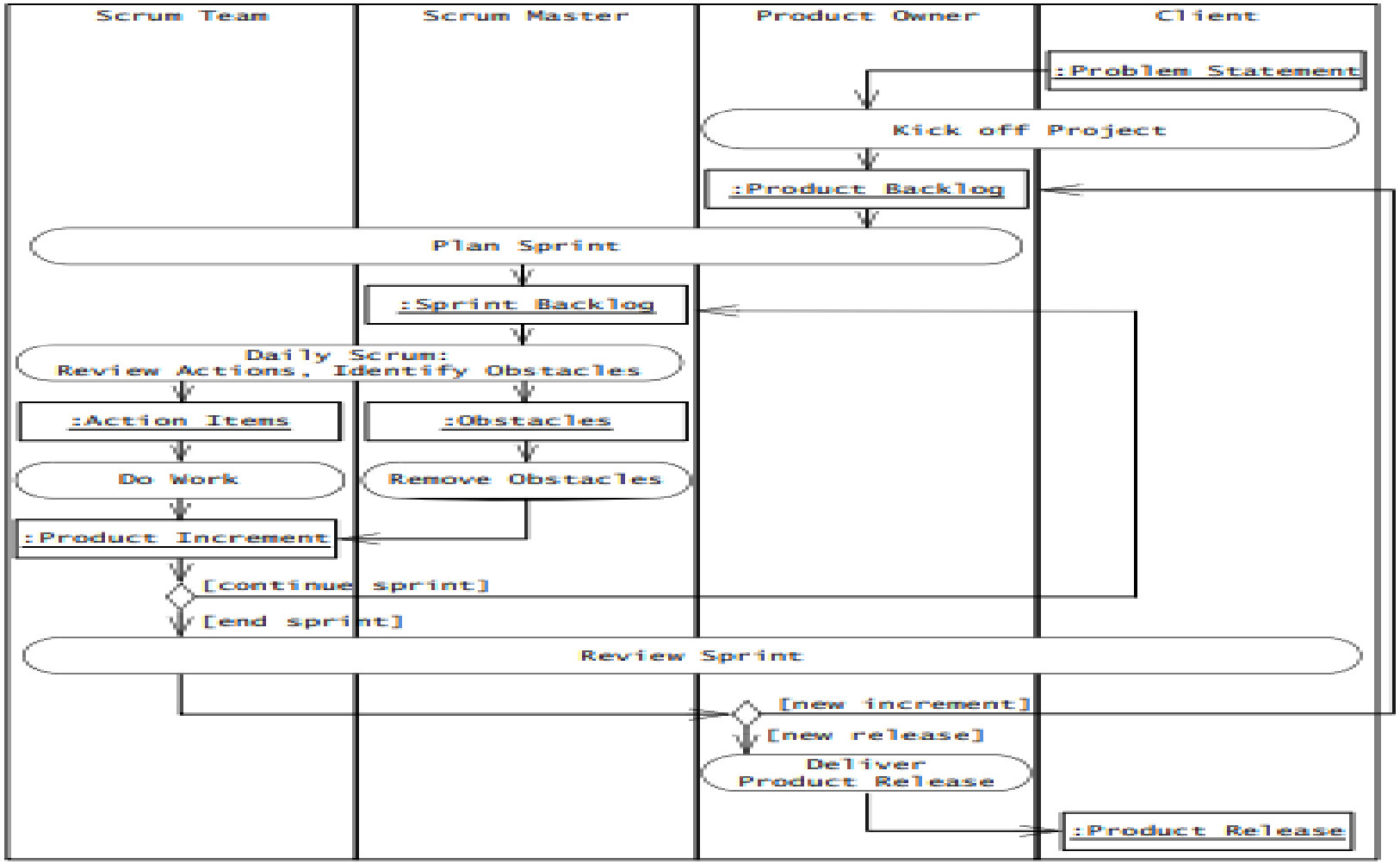


Figure 14-16 Overview of a Scrum process (UML activity diagram).

Cont….

- Each iteration is considered as a short time "frame" in the Agile process model, which typically lasts from one to four weeks.
- The division of the entire project into smaller parts helps to minimize the project risk and to reduce the overall project delivery time requirements.
- Each iteration involves a team working through a full software development life cycle including planning, requirements analysis, design, coding, and testing before a working product is demonstrated to the client.

Cont..

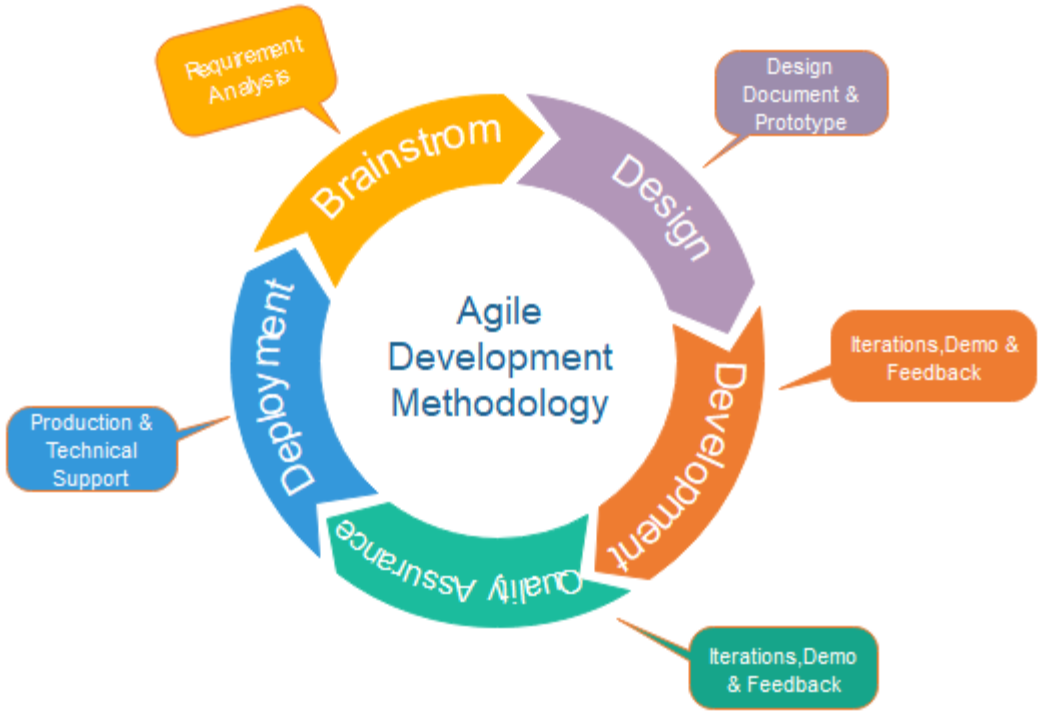


Fig. Agile Model

…Phases of Agile Model:

- **Requirements gathering:** must define the requirements.
- **Design the requirements:** you have identified the project, work with stakeholders to define requirements. You can use the user flow diagram or the high-level UML diagram
- **Construction/ iteration:** Designers and developers start working on their project, which aims to deploy a working product.
- **Testing/ Quality assurance:** Quality Assurance team examines the product's performance and looks for the bug.
- **Deployment:** the team issues a product for the user's work environment.
- **Feedback:** the team receives feedback about the product and works through the feedback.

Cont…

- When to use the Agile Model?
  - When frequent changes are required.
  - When a highly qualified and experienced team is available.
  - When a customer is ready to have a meeting with a software team all the time.
  - When project size is small.
- Advantage(Pros) of Agile Method:
  - Frequent Delivery
  - Face-to-Face Communication with clients.
  - Efficient design and fulfils the business requirement.
  - Anytime changes are acceptable.
  - It reduces total development time.
- Disadvantages(Cons) of Agile Model:
  - Due to the shortage of formal documents, it creates confusion and crucial decisions taken throughout various phases can be misinterpreted at any time by different team members.
  - Due to the lack of proper documentation, once the project completes and the developers allotted to another project, maintenance of the finished project can become a difficulty.

Model Selection parameters

- Selection Process parameters plays an important role in software development as it helps to choose the best suitable software life cycle model. Following are the parameters which should be used to select a SDLC.
- **1. Requirements characteristics :**
  - Reliability of Requirements
  - How often the requirements can change
  - Types of requirements
  - Number of requirements
  - Can the requirements be defined at an early stage
  - Requirements indicate the complexity of the system
- **2. Development team :**
  - Team size
  - Experience of developers on similar type of projects
  - Level of understanding of user requirements by the developers
  - Environment
  - Domain knowledge of developers
  - Experience on technologies to be used
  - Availability of training

Cont..

3. User involvement in the project :

- Expertise of user in project
- Involvement of user in all phases of the project
- Experience of user in similar project in the past

- **4. Project type and associated risk :**
  - Stability of funds
  - Tightness of project schedule
  - Availability of resources
  - Type of project
  - Size of the project
  - Expected duration for the completion of project
  - Complexity of the project
  - Level and the type of associated risk

1.3. Object oriented system development methodology.

- Object-Oriented Software Engineering (OOSE) is a software design technique that is used in software design in object-oriented programming.
- Object-oriented programming (OOP) is **a computer programming model that organizes software design around data, or objects, rather than functions** and logic. ... OOP focuses on the objects that developers want to manipulate rather than the logic required to manipulate them.

Cont…

- As the name suggests, Object-Oriented Programming or OOPs refers to languages that use objects in programming.
- Object-oriented programming aims to implement real-world entities like inheritance, abstraction, polymorphism, etc in programming.
- The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

Cont…

- The fundamental concepts of the object-oriented world:
  - **Objects** – Objects represent an entity and the basic building block.
  - **Class** – Class is the blue print of an object.
  - **Abstraction** – is used to hide unnecessary information and display only necessary information to the users interacting.
  - **Encapsulation** – Encapsulation is the mechanism of binding the data together and hiding them from the outside world.
  - **Inheritance** – Inheritance is the mechanism of making new classes from existing ones.
  - **Polymorphism** – It defines the mechanism to exists in different forms.

Object oriented system development methodology.

- Object oriented development offers a different model from traditional software development approach(which based on the function of and the procedure).
- In simplified terms, object oriented systems development is the way to develop software by building self-containing modules or objects that can be easily replaced, modified and reused.
- In o-o environment, software is collection of discrete objects that encapsulate their data as well as the functionality to model real world object.
- In o-o system, everything is object and each object responsible for it self.

BENEFITS OF OBJECT ORIENTATION

- **Faster development,**
- **Reusability,**
- **Increased quality**
- **Modeling the real world and provides us with the stronger the equivalence of the real world's entities (objects)**
- Raising the level of abstraction to the point where **application can be implemented in the same terms as they are described.**

WHY OBJECT ORIENTATION

- OO Methods enables to develop **set of objects that work** together software similar to traditional techniques.
- It adapts to
  - **Changing requirements**
  - **Easier to maintain**
  - **More robust**
  - **Promote greater design**
  - **Code reuse**
- Others
  - **Higher level of abstraction**
  - **Seamless transition among different phases of software development.**
  - **Encouragement of good programming technique.**
  - **Promotion of reusability.**

OBJECT BASICS

- **Goals:** In object oriented method the developer should:
  - Define Objects and classes
  - Describe objects, methods, attributes and how objects respond to messages,
  - Define Polymorphism, Inheritance, data abstraction, encapsulation,
  - Describe objects relationships,
  - Describe object persistence,

OBJECT-ORIENTED APPROACH

- OO approach is more like **creating a lot of helpers**
- take on an **active role, a spirit**, that form a community whose **interactions become the application**.
- Reusable
- Modified
- Replaced

EXAMPLES OF OBJECT ORIENTED SYSTEMS

- In OO system , **“everything is object”.**
- **A spreadsheet cell, bar chart, title in bar chart, report,**
- **numbers, arrays, records, fields, files, forms, an invoice, etc.**
- **A window object** is responsible for things like **opening, sizing, and closing itself.**
- A **chart object** is responsible for things like **maintaining data and labels even for drawing itself.**

WHAT IS AN OBJECT AND CLASS?

- The concepts of objects and classes are intrinsically linked with each other and form the foundation of o-o paradigm.
- An object is an entity, it knows things (has attributes), It does things (provides services or has methods).
- Identity that distinguishes it from other objects in the system.
- State that determines the characteristic properties of an object as well as the values of the properties that the object holds.
- Behavior that represents externally visible activities performed by an object in terms of changes in its state.

Class

- A class represents a collection of objects having same characteristic properties that exhibit common behavior.
- Creation of an object as a member of a class is called instantiation.
- Thus, object is an instance of a class.
- Example: **Circle** – a class
  - x, to the center
  - a, to denote the radius of the circle
- Some of its operations can be defined as follows: findArea(), method to calculate area
  - findCircumference(), method to calculate circumference

OBJECT'S ATTRIBUTES

- **Attributes represented by data type.**
- **They describe objects states.**
- **In the Car example the car's attributes are: color, manufacturer, cost, owner, model, etc.**

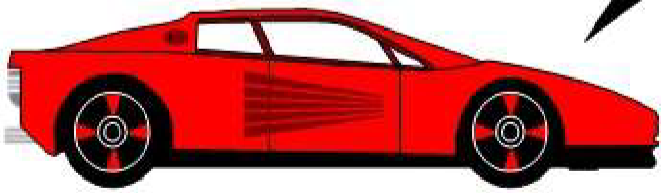
OBJECT’S METHODS

- **Methods define objects behavior and specify the way in which an Object’s data are manipulated.**
- **In the Car example the car’s methods are: drive it, lock it, tow it, carry passenger in it.**

IT KNOWS THINGS (ATTRIBUTES)

ATTRIBUTES

I am a Car.
I know my color, manufacturer, cost, owner and model.

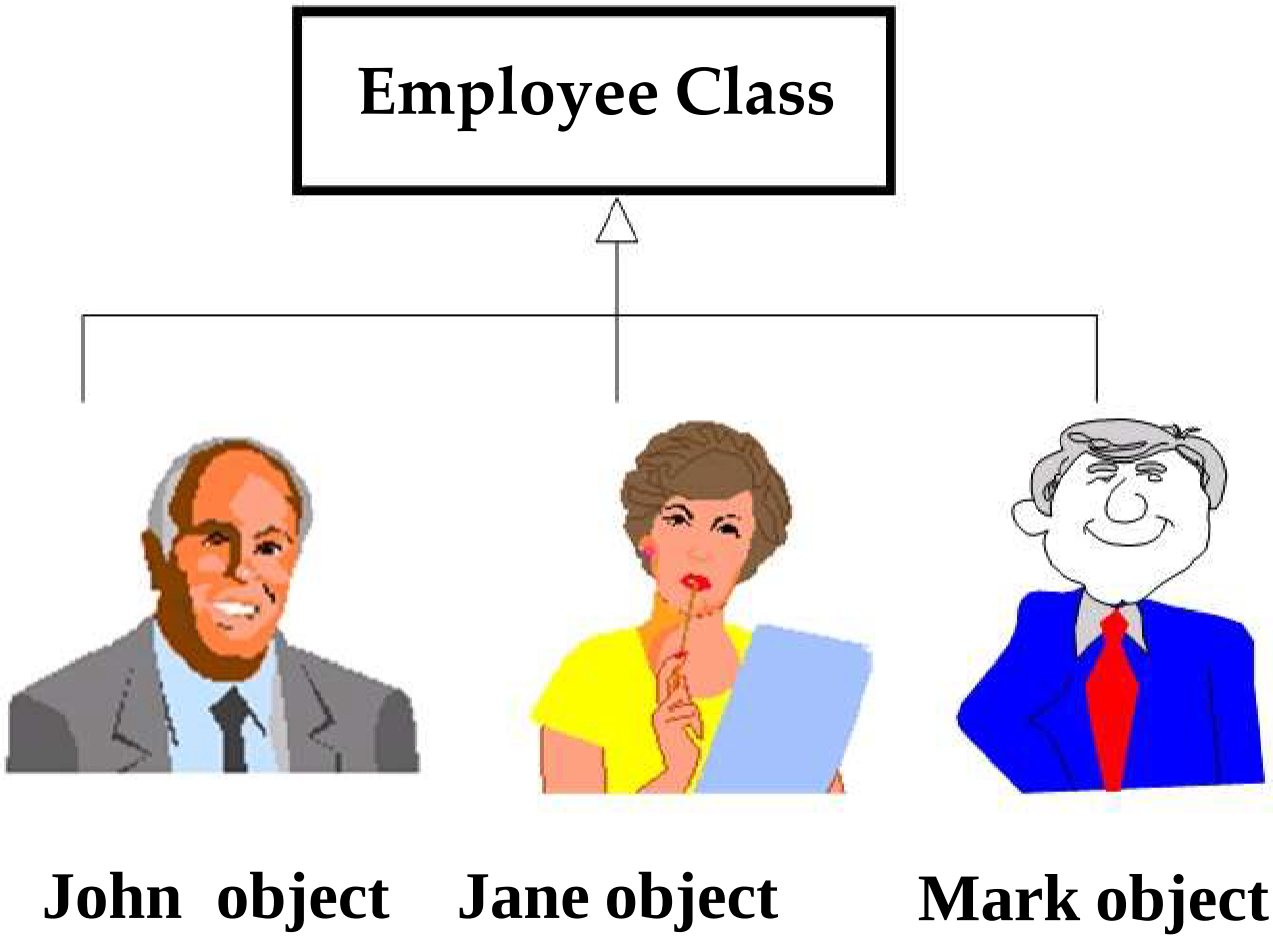


IT DOES THINGS (METHODS)

METHODS

OBJECTS ARE GROUPED IN CLASSES

- The role of a class is to **define the attributes and methods** (the state and behavior) **of its instances.**
- Used to **distinguish one type of object from the other.**
- Set of objects**, that share common methods, structure, behavior.**
- **Single object** is simply an **instance of class.**
- The **class car**, for example, **defines the property color.** Each individual car (object) will have a **value for this property, such as "maroon," "yellow" or "white."**


Employee Class
John object
Jane object
Mark object

Cont..

- THANK YOU !